

INSULOG / SEGURANÇA DE APLICAÇÕES

Relatório de Auditoria de Segurança — Insulog

Revisão de código • Evidências rastreáveis • Plano de correção

17/09/2026

ESCOPO AUDITADO

Frontend Flutter/Dart e backend Node.js/Express/MySQL dos clones locais. Código próprio, configurações, CI, documentação, histórico Git alcançável e bundle web gerado nesta revisão.

Frontend: ee623bbf1b0a538e140141388dab23982705677b
Backend: feb285d76d9b059e5e2f440c55472a95abbd06ca

NOTA METODOLÓGICA

As cinco categorias foram adaptadas a filtros manuais de dono no MySQL, autorização das rotas Express, IDs em path/query/body, segredos em arquivos/histórico/bundle e sinks de HTML/JavaScript no Flutter web e backend. Achados adicionais de logs, armazenamento e transporte são contados separadamente.

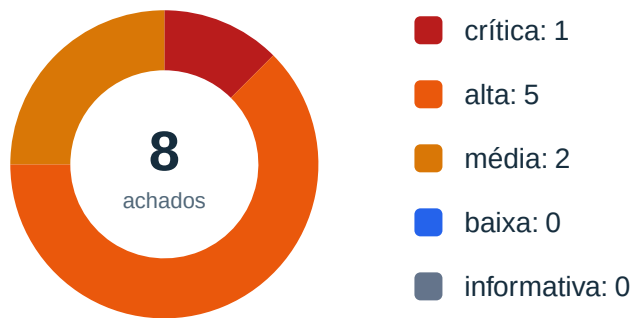
Revisão estática com reprodução local usando banco simulado. Não houve acesso a dados reais, uso de credenciais históricas, alteração da aplicação ou publicação de issues.

Resumo executivo

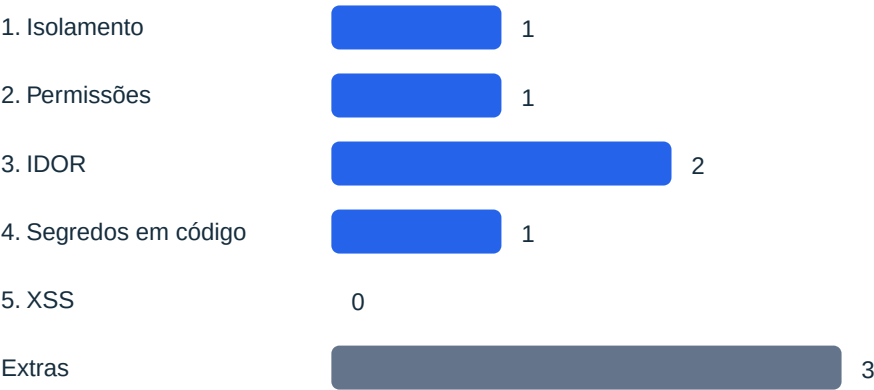
A fronteira de autenticação termina no login. A API permite consultar dados de outros pacientes e alterar contas e registros sem identidade validada. A prioridade é impedir acesso anônimo e vincular toda operação ao dono autorizado.

8 achados: 1 crítico • 5 altos • 2 médios • 0 baixos • 0 informativos

Distribuição por severidade



Distribuição por categoria



Contagem por causa/escopo acionável, sem duplicar listagens por ID entre isolamento e IDOR. Cinco achados nas categorias solicitadas e três extras. Nenhum XSS confirmado.

Stack, escopo e limites

Camada	Deteção / evidência
Frontend	Dart (SDK declarado ^3.11.4), Flutter, package:http, shared_preferences, share_plus, flutter_svg. pubspec.yaml:1-26; suporte web e runners nativos.
Backend	JavaScript CommonJS, Express ^5.2.1, mysql2 ^3.20.0; SQL manual, sem ORM/query builder. package.json:9-23.
Autenticação	Login com script e fallback legado; resposta somente com usuário. Sem sessão, JWT, cookie de autenticação ou middleware de autorização no código.
Isolamento	Filtros manuais id_usuario e join de registroinsulina → registroglicose. Sem identidade autenticada nem mecanismo de tenant. Relação paciente/médico é persistida, mas não autoriza consultas.
Relatórios	PDFKit ^0.20.2 e ExcelJS ^4.4.0; sem templates de HTML/e-mail.
Deploy/CI	GitHub Actions (Flutter Tests, API Tests), Gradle e projetos de plataforma. Não encontrados Dockerfile, compose, Helm, Terraform ou configuração de gateway nos arquivos próprios atuais/históricos inspecionados.

F/ identifica a raiz insulog-mobile-front-clone; B/ identifica ../insulog-back-clone. Números de linha referem-se aos HEADs da capa, exceto evidências históricas explicitamente marcadas.

Cobertura: 217 arquivos próprios versionados no frontend e 54 no backend inventariados; 17 + 19 commits e 302 + 124 blobs textuais próprios examinados por padrões. Arquivos de dependências, binários e runners gerados não receberam auditoria integral de implementação. Todos os controllers, services, repositories e routers próprios do backend foram lidos.

Limites: sem validação do banco/deploy real, constraints completas, credenciais atuais, gateway, ACL de rede, reflogs/objetos perdidos ou remotos não baixados. Busca de segredos por padrões e revisão manual não garante ausência absoluta. Este relatório não é auditoria de CVEs de dependências.

1. Banco sem tranca

Aplicável: MySQL/mysql2, isolamento manual por id_usuario. F01. Sem identidade autenticada, os filtros não constituem autorização.

2. Permissão no navegador

Não há gates de papel no Flutter. Mapeamento para escrita privilegiada na API: F02. Atualização de conta/papel é F03. O cadastro fixa paciente no app, mas backend aceita medico com CRM não vazio; sem regra de negócio de aprovação ou privilégio implementado, isso não é contado como escalada verificada.

3. IDOR

Todos os 48 handlers dos routers revisados estaticamente, mais GET / e middleware OPTIONS. 47 handlers exercitados; um sombreado. F03/F04 tratam objetos; consultas coletivas por ID de usuário estão agrupadas em F01 para evitar dupla contagem.

4. Chaves expostas

F05: .env no histórico. Sem chave hardcoded ativa confirmada no HEAD, CI, scripts, documentação ou bundle web compilado. IP privado e ID de médico não são segredos. Logs, armazenamento e HTTP são extras, não hardcoded.

5. XSS

Aplicável ao alvo web e saídas; nenhum achado confirmado. Não foram encontrados sinks de HTML/JS alimentados por usuário. Templates HTML/e-mail não existem neste backend. PDFKit/ExcelJS não renderizam HTML como página web.

Pontos fortes e riscos centrais

✓ Senhas novas recebem hash com salt aleatório e comparação resistente a timing.

O fallback legado de comparação de texto puro em passwordService.js:46–47 existe; a migração de registros antigos não foi demonstrada. Isso não protege a listagem F01 nem o log anterior ao hash F06.

B/src/services/passwordService.js:6–9
B/src/services/passwordService.js:27–34
B/src/services/userService.js:70–74

✓ Consultas usam parâmetros; trechos dinâmicos de LIMIT/ordenação são normalizados.

Verificados os oito repositories e seus services. Nenhuma SQL injection confirmada nos caminhos revisados. Parâmetros evitam injeção, mas não garantem posse.

B/src/repositories/userRepository.js:62–66
B/src/services/registroGlicoseService.js:27–35
B/src/services/registroInsulinaService.js:25–33
B/src/repositories/registroGlicoseRepository.js:301–310

✓ Exportação valida ID, datas e formato, com headers e conferência do arquivo no app.

O filtro de dono ainda depende do ID controlável pelo cliente (F01).

B/src/services/exportacaoService.js:92–99
B/src/services/exportacaoService.js:107–111
B/src/controllers/exportacaoController.js:51–52
B/src/controllers/exportacaoController.js:63–67
F/lib/services/api/report_export_service.dart:96–110

✓ Fluxos de texto revisados não interpretam HTML controlado pelo usuário.

Flutter usa Text/TextField; API usa JSON e exporta PDF/XLSX. Não há templates HTML/e-mail, markdown, WebView ou sinks de HTML/JavaScript controlados pelo usuário. escape-html é transitivo do Express; _sanitizePayload mascara logs e não é sanitizador HTML.

F/lib/widgets/home/home_header_widget.dart:24–26
B/src/reports/pdfReport.js:316–318
B/src/controllers/authController.js:5–6

✓ Logs do cliente mascaram campos senha/password/token recursivamente.

Proteção restrita ao cliente e a esses campos; não cobre os logs integrais do servidor.

F/lib/services/api/api_service.dart:274–288

✓ Segredos atuais do banco são recebidos do ambiente e CI backend usa leitura mínima.

Não há fallback literal de senha no HEAD. Startup não valida/rejeita credenciais fracas: usa env e apenas registra falha de conexão em database.js:20–27. Não foi confirmado default secreto ativo.

B/src/config/database.js:8–13
B/.github/workflows/api-tests.yml:11–12

Pontos fracos

Identidade local não autentica chamadas HTTP; queries globais e IDs controláveis quebram confidencialidade e integridade. Senhas também escapam por logs, persistência local e rede sem TLS. A senha de banco retirada do HEAD ainda pode ser recuperada pelo histórico.

Tabela de achados por categoria

Cada linha aponta um trecho exato; as páginas seguintes trazem código, impacto, condições e correção. Os chips indicam a severidade do achado ao qual a evidência pertence.

1. Isolamento

Severidade	Arquivo:linha	Descrição
ALTA	B/src/app.js:96-104	F01 — Leitura global de usuários, dados de saúde e relatórios sem identidade validada
ALTA	B/src/services/authService.js:21-28	F01 — evidência complementar
ALTA	B/src/repositories/userRepository.js:4-9	F01 — evidência complementar
ALTA	B/src/controllers/userController.js:3-6	F01 — evidência complementar
ALTA	B/src/repositories/ registroGlicoseRepository.js:6-9	F01 — evidência complementar
ALTA	B/src/repositories/ registroInsulinaRepository.js:4-9	F01 — evidência complementar
ALTA	B/src/repositories/alarmeRepository.js:14-19	F01 — evidência complementar
ALTA	B/src/repositories/ configuracaoRepository.js:4-9	F01 — evidência complementar
ALTA	B/src/repositories/ exportacaoRepository.js:4-9	F01 — evidência complementar
ALTA	B/src/controllers/ registroGlicoseController.js:14-27	F01 — evidência complementar
ALTA	B/src/controllers/ registroGlicoseController.js:44-61	F01 — evidência complementar
ALTA	B/src/services/registroGlicoseService.js:38-51	F01 — evidência complementar
ALTA	B/src/services/registroInsulinaService.js:36-49	F01 — evidência complementar
ALTA	B/src/controllers/alarmeController.js:12-16	F01 — evidência complementar
ALTA	B/src/controllers/exportacaoController.js:54-60	F01 — evidência complementar
ALTA	B/src/repositories/ exportacaoRepository.js:108-111	F01 — evidência complementar

Severidade	Arquivo:linha	Descrição
ALTA	F/lib/services/api/api_service.dart:18-21	F01 — evidência complementar
ALTA	F/lib/services/api/report_export_service.dart:58-70	F01 — evidência complementar

2. Permissões

Severidade	Arquivo:linha	Descrição
ALTA	B/src/routes/periodoRoutes.js:8-10	F02 — Catálogos globais aceitam criação, edição e exclusão anônimas
ALTA	B/src/routes/typeInsuRoutes.js:8-10	F02 — evidência complementar
ALTA	B/src/services/periodoService.js:40-46	F02 — evidência complementar
ALTA	B/src/services/typeInsuService.js:40-46	F02 — evidência complementar
ALTA	B/src/repositories/periodoRepository.js:55-56	F02 — evidência complementar
ALTA	B/src/repositories/typeInsuRepository.js:37-38	F02 — evidência complementar

3. IDOR

Severidade	Arquivo:linha	Descrição
CRÍTICA	B/src/routes/userRoutes.js:8-14	F03 — Atualização de conta por ID permite trocar a senha de outro usuário
CRÍTICA	B/src/controllers/userController.js:24-27	F03 — evidência complementar
CRÍTICA	B/src/controllers/userController.js:35-37	F03 — evidência complementar
CRÍTICA	B/src/controllers/userController.js:55-57	F03 — evidência complementar
CRÍTICA	B/src/services/userService.js:78-92	F03 — evidência complementar
CRÍTICA	B/src/repositories/userRepository.js:134-138	F03 — evidência complementar
CRÍTICA	B/src/repositories/userRepository.js:18-30	F03 — evidência complementar

Severidade	Arquivo:linha	Descrição
ALTA	B/src/repositories/ registroGlicoseRepository.js:53-54	F04 — Registros, alarmes e preferências usam IDs e donos fornecidos pelo cliente
ALTA	B/src/services/registroGlicoseService.js:125-130	F04 — evidência complementar
ALTA	B/src/repositories/ registroGlicoseRepository.js:198-205	F04 — evidência complementar
ALTA	B/src/repositories/ registroGlicoseRepository.js:278-289	F04 — evidência complementar
ALTA	B/src/repositories/ registroInsulinaRepository.js:14-15	F04 — evidência complementar
ALTA	B/src/repositories/ registroInsulinaRepository.js:52-53	F04 — evidência complementar
ALTA	B/src/repositories/ registroInsulinaRepository.js:80-81	F04 — evidência complementar
ALTA	B/src/repositories/ registroInsulinaRepository.js:102-103	F04 — evidência complementar
ALTA	B/src/repositories/alarmeRepository.js:32-33	F04 — evidência complementar
ALTA	B/src/repositories/alarmeRepository.js:47-48	F04 — evidência complementar
ALTA	B/src/repositories/alarmeRepository.js:80-81	F04 — evidência complementar
ALTA	B/src/repositories/alarmeRepository.js:107-108	F04 — evidência complementar
ALTA	B/src/repositories/ configuracaoRepository.js:14-15	F04 — evidência complementar
ALTA	B/src/repositories/ configuracaoRepository.js:29-30	F04 — evidência complementar
ALTA	B/src/repositories/ configuracaoRepository.js:58-59	F04 — evidência complementar
ALTA	B/src/repositories/ configuracaoRepository.js:81-82	F04 — evidência complementar
ALTA	B/src/repositories/ exportacaoRepository.js:14-15	F04 — evidência complementar
ALTA	B/src/repositories/ exportacaoRepository.js:29-30	F04 — evidência complementar

Severidade	Arquivo:linha	Descrição
ALTA	B/src/repositories/exportacaoRepository.js:57-58	F04 — evidência complementar
ALTA	B/src/repositories/exportacaoRepository.js:79-80	F04 — evidência complementar

4. Segredos em código

Severidade	Arquivo:linha	Descrição
MÉDIA	B/.env @ 147015e e 9ec3cab:3-4	F05 — Senha do MySQL permanece recuperável no histórico Git
MÉDIA	B/src/config/database.js:4-13	F05 — evidência complementar

Extra: logs

Severidade	Arquivo:linha	Descrição
ALTA	B/src/app.js:39-48	F06 — Logs da API registram senhas e respostas sensíveis sem mascaramento
ALTA	B/src/app.js:78-85	F06 — evidência complementar
ALTA	B/src/controllers/userController.js:43-47	F06 — evidência complementar
ALTA	B/src/services/userService.js:37-40	F06 — evidência complementar

Extra: armazenamento

Severidade	Arquivo:linha	Descrição
MÉDIA	F/lib/states/login_form_state.dart:54-59	F07 — App persiste a senha reutilizável em SharedPreferences
MÉDIA	F/lib/services/local/saved_login_service.dart:25-28	F07 — evidência complementar
MÉDIA	F/lib/services/local/saved_login_service.dart:31-35	F07 — evidência complementar
MÉDIA	F/lib/states/login_form_state.dart:111-115	F07 — evidência complementar

Extra: transporte

Severidade	Arquivo:linha	Descrição
ALTA	F/lib/services/local/api_ip_service.dart:18-26	F08 — Login e dados de saúde trafegam em HTTP sem TLS
ALTA	F/lib/services/api/auth_service.dart:14-17	F08 — evidência complementar
ALTA	F/lib/services/api/api_service.dart:30-32	F08 — evidência complementar
ALTA	F/android/app/src/main/AndroidManifest.xml:13-15	F08 — evidência complementar
ALTA	B/src/server.js:13-16	F08 — evidência complementar

5. XSS

Nenhum achado confirmado; resultado e evidências positivas registrados no escopo e nos pontos fortes.

F01 — Leitura global de usuários, dados de saúde e relatórios sem identidade validada

ALTA

PROBLEMA E EXPLORABILIDADE

O login apenas devolve os dados do usuário. Nenhum middleware estabelece identidade autenticada nas rotas. As listagens retornam todos os donos; GET /usuarios também devolve a coluna senha por SELECT *. Nas buscas por usuário, dashboard, histórico e exportação, o filtro id_usuario usa diretamente path/query do chamador. É possível consultar outro paciente, inclusive por nome/e-mail nas listagens de glicose/insulina. Não existe RLS ou middleware de tenant: há filtros manuais de dono, sem vínculo com autenticação.

IMPACTO

Exposição de nomes, e-mails, hashes de senha, medições, doses, alarmes, preferências e relatórios de outros pacientes. Se existirem senhas legadas em texto puro, também saem na listagem; a existência desses registros não foi comprovada.

CONDIÇÕES E VALIDAÇÃO

Acesso de rede à API e dados existentes. Não exige login, flag ou configuração adicional no código. Não foi inspecionado um eventual gateway externo. Teste local confirmou resposta 200 e campo senha com hash; banco simulado.

Evidências por arquivo e linha

B/src/app.js:96-104

```
96: app.use('/', authRoutes)
97: app.use('/usuarios', userRoutes)
98: app.use('/tipos-insulina', typeInsuRoutes)
99: app.use('/periodos', periodoRoutes)
100: app.use('/registros-glicose', registroGlicoseRoutes)
101: app.use('/registros-insulina', registroInsulinaRoutes)
102: app.use('/exportacoes', exportacaoRoutes)
103: app.use('/configuracoes', configuracaoRoutes)
104: app.use('/alarmes', alarmeRoutes)
```

B/src/services/authService.js:21-28

```
21:   return {
22:     user: {
23:       id: user.id_usuario,
24:       username: user.nome,
25:       email: user.email,
26:       tipo_usuario: user.tipo_usuario
27:     }
28:   }
```

B/src/repositories/userRepository.js:4-9

```
4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT * FROM usuario ORDER BY id_usuario ASC'
7:   )
8:
9:   return rows
```

B/src/controllers/userController.js:3-6

```
3: async function index(req, res, next) {
4:   try {
5:     const users = await userService.listUsers()
6:     return res.status(200).json(users)
```

B/src/repositories/registroGlicoseRepository.js:6-9

```
6:   `SELECT rg.id_registro, rg.id_usuario, rg.nivel_glicose, rg.data_hora, p.descricao AS periodo
7:     FROM registroglicose rg
8:     LEFT JOIN periodo p ON p.id_periodo = rg.id_periodo
9:     ORDER BY rg.id_registro ASC`
```

B/src/repositories/registroInsulinaRepository.js:4-9

```
4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT id_registro_insulina, id_registro, id_tipo_insulina, unidade_insulina FROM registroins
  ulina ORDER BY id_registro_insulina ASC'
7:   )
8:
9:   return rows
```

B/src/repositories/alarmeRepository.js:14-19

```
14: async function findAll() {
15:   const [rows] = await db.execute(
16:     'SELECT id_alarme, id_usuario, data_hora, id_periodo, id_registro, dias_semana, ativo, tem_so
  m, tem_vibracao FROM alarme ORDER BY id_alarme ASC'
17:   )
18:
19:   return rows.map(formatAlarme)
```

B/src/repositories/configuracaoRepository.js:4-9

```
4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT id_configuracao, id_usuario, idioma, tema, notificacoes FROM configuracao ORDER BY id_
  configuracao ASC'
7:   )
8:
9:   return rows
```

B/src/repositories/exportacaoRepository.js:4-9

```
4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT id_exportacao, id_usuario, data, descricao FROM exportacao ORDER BY id_exportacao ASC'
7:   )
8:
9:   return rows
```

B/src/controllers/registroGlicoseController.js:14-27

```
14:   const { id_usuario } = req.params
15:   const { dataInicio, dataFim, quantidade } = req.query
16:
17:   if (quantidade && quantidade !== 'null') {
18:     const registrosGlicose = await registroGlicoseService.getRegistrosGlicoseByUserId(id_usuari
  o, quantidade)
19:     return res.status(200).json(registrosGlicose)
20:   }
21:
22:   if (dataInicio || dataFim) {
23:     const dashboardDados = await registroGlicoseService.getDashboardDados(id_usuario, dataInici
```

```
    o, dataFim)
24:     return res.status(200).json(dashboardDados)
25:   }
26:
27:   const registrosGlicose = await registroGlicoseService.getRegistrosGlicoseById(id_usuario,
    quantidade)
```

B/src/controllers/registroGlicoseController.js:44-61

```
44: async function getDashboard(req, res, next) {
45:   try {
46:     const { id_usuario, dataInicio, dataFim } = req.query
47:     const dashboardDados = await registroGlicoseService.getDashboardDados(id_usuario, dataInicio,
    dataFim)
48:     return res.status(200).json(dashboardDados)
49:   } catch (error) {
50:     next(error)
51:   }
52: }
53:
54: async function getHistorico(req, res, next) {
55:   try {
56:     const { id_usuario } = req.params
57:     const { dataInicio, dataFim } = req.query
58:     const historico = await registroGlicoseService.getHistorico(
59:       id_usuario,
60:       dataInicio,
61:       dataFim
```

B/src/services/registroGlicoseService.js:38-51

```
38: async function getRegistrosGlicoseById(usuario, quantidade) {
39:   const usuarioId = Number(usuario)
40:   const useId = Number.isNaN(usuarioId)
41:   ? await userRepository.findById(usuario)
42:   : { id_usuario: usuarioId }
43:
44:   if (!useId) {
45:     const error = new Error('Usuario nao encontrado')
46:     error.statusCode = 400
47:     throw error
48:   }
49:
50:   const quantidadeRegistros = normalizarQuantidade(quantidade)
51:   const registrosGlicose = await registroGlicoseRepository.findById(useId.id_usuario, quantid
    adeRegistros)
```

B/src/services/registroInsulinaService.js:36-49

```
36: async function getRegistrosInsulinaById(usuario, quantidade) {
37:   const usuarioId = Number(usuario)
38:   const useId = Number.isNaN(usuarioId)
39:   ? await userRepository.findById(usuario)
40:   : { id_usuario: usuarioId }
41:
42:   if (!useId) {
43:     const error = new Error('Usuario nao encontrado')
44:     error.statusCode = 400
45:     throw error
46:   }
47:
48:   const quantidadeRegistros = normalizarQuantidade(quantidade)
49:   const registrosInsulina = await registroInsulinaRepository.findById(useId.id_usuario, quant
```

```
idadeRegistros)
```

B/src/controllers/alarmeController.js:12-16

```
12: async function showByUsuarioId(req, res, next) {
13:   try {
14:     const { usuarioId } = req.params
15:     const alarmes = await alarmeService.getAlarmesByUsuarioId(usuarioId)
16:     return res.status(200).json(alarmes)
```

B/src/controllers/exportacaoController.js:54-60

```
54:   const { id_usuario, dataInicio, dataFim, formato } = req.query
55:   const arquivo = await exportacaoService.gerarRelatorio({
56:     idUsuario: id_usuario,
57:     dataInicio,
58:     dataFim,
59:     formato
60:   })
```

B/src/repositories/exportacaoRepository.js:108-111

```
108:   WHERE rg.id_usuario = ? AND rg.data_hora >= ?
109:         AND rg.data_hora < DATE_ADD(?, INTERVAL 1 DAY)
110:   ORDER BY rg.data_hora ASC, ri.id_registro_insulina ASC`,
111:   [idUsuario, dataInicio, dataFim]
```

F/lib/services/api/api_service.dart:18-21

```
18: Map<String, String> get headers => {
19:   'Content-Type': 'application/json',
20:   'Accept': 'application/json',
21:   };
```

F/lib/services/api/report_export_service.dart:58-70

```
58:   final uri = Uri.parse('${await _baseUrl()}/exportacoes/relatorio')
59:   .replace(
60:     queryParameters: {
61:       'id_usuario': '$userId',
62:       'dataInicio': firstDay,
63:       'dataFim': lastDay,
64:       'formato': format.name,
65:     },
66:   );
67:   final get = _client?.get ?? http.get;
68:   final response = await get(
69:     uri,
70:     headers: {'Accept': format.mimeType},
```

CORREÇÃO PROPOSTA

Criar sessão/token validado no servidor e negar acesso por padrão. Derivar o dono da identidade e, para médicos, validar o vínculo autorizado com o paciente. Aplicar o mesmo escopo a listagens, dashboard, histórico e exportação. Substituir SELECT * por projeção sem senha. Remover listagens globais desnecessárias.

F02 — Catálogos globais aceitam criação, edição e exclusão anônimas

ALTA

PROBLEMA E EXPLORABILIDADE

POST, PUT e DELETE de /periodos e /tipos-insulina chamam diretamente os controllers. Os services verificam campos/existência, mas nenhum privilégio. As tabelas são catálogos compartilhados, sem coluna de dono. Não há isAdmin/canEdit/role nem UI de manutenção desses catálogos no Flutter; portanto não se afirma um gate de papel escondido no navegador. A falha verificada é a ausência de autorização no servidor para escrita global.

IMPACTO

Qualquer chamador altera nomes/tipos/periodos usados por todos os usuários, cria entradas arbitrárias e remove entradas não referenciadas.

CONDIÇÕES E VALIDAÇÃO

API alcançável. Edição exige ID existente e campo nome/descricao; exclusão pode ser impedida por foreign keys se o catálogo estiver em uso. A edição anônima foi exercitada com banco simulado.

Evidências por arquivo e linha

B/src/routes/periodoRoutes.js:8-10

```
8: router.post('/', periodoController.create)
9: router.put('/:id', periodoController.update)
10: router.delete('/:id', periodoController.deleteById)
```

B/src/routes/typeInsuRoutes.js:8-10

```
8: router.post('/', typeInsuController.create)
9: router.put('/:id', typeInsuController.update)
10: router.delete('/:id', typeInsuController.deleteById)
```

B/src/services/periodoService.js:40-46

```
40: if (!data.descricao) {
41:   const error = new Error('0 campo descricao é obrigatório')
42:   error.statusCode = 400
43:   throw error
44: }
45:
46: return await periodoRepository.update(id, { descricao: data.descricao })
```

B/src/services/typeInsuService.js:40-46

```
40: if (!tipoInsulina.nome) {
41:   const error = new Error('0 campo nome é obrigatório')
42:   error.statusCode = 400
43:   throw error
44: }
45:
46: return await typeInsuRepository.updateById(id, tipoInsulina)
```

B/src/repositories/periodoRepository.js:55-56

```
55: 'UPDATE periodo SET descricao = ? WHERE id_periodo = ?',
56: [descricao, id]
```

B/src/repositories/typeInsuRepository.js:37-38

```
37:      'UPDATE tipoinsulina SET nome = ? WHERE id_tipo_insulina = ?',  
38:      [nome, id]
```

CORREÇÃO PROPOSTA

Exigir identidade e permissão explícita de manutenção dos catálogos em todas as três operações. Se não houver função administrativa no produto, remover/desabilitar as rotas de escrita e manter carga controlada de catálogos.

F03 — Atualização de conta por ID permite trocar a senha de outro usuário

CRÍTICA

PROBLEMA E EXPLORABILIDADE

PUT /usuarios/:id aceita nome, email, senha, tipo_login e tipo_usuario do body, calcula o hash e executa UPDATE usuario WHERE id_usuario = ?. Não verifica sessão, posse, senha atual nem permissão sobre o papel. GET e DELETE da conta também usam somente o ID. Conhecendo um ID e os campos retornados pelas listagens, o chamador pode substituir a senha da vítima.

IMPACTO

Tomada de conta, alteração de identidade/papel e exclusão de contas. A alteração de papel é persistida, mas não foi demonstrado um privilégio adicional de médico, pois o projeto não implementa esse controle.

CONDIÇÕES E VALIDAÇÃO

API alcançável, usuário existente e body com os campos obrigatórios. Restrições reais do banco podem impedir valores inválidos ou exclusão com referências. Teste com services/repository reais e banco simulado confirmou hash da nova senha e ID escolhido.

Evidências por arquivo e linha

B/src/routes/userRoutes.js:8-14

```
8: router.get('/', userController.index);
9: router.get('/:id', userController.show);
10: router.get('/:tipo_usuario', userController.showByType);
11:
12: router.post('/', userController.create);
13: router.delete('/:id', userController.deleteById);
14: router.put('/:id', userController.update);
```

B/src/controllers/userController.js:24-27

```
24:   const { id } = req.params
25:   const user = await userService.getUserById(id)
26:
27:   return res.status(200).json(user)
```

B/src/controllers/userController.js:35-37

```
35:   const { id } = req.params
36:   await userService.deleteById(id)
37:   return res.status(204).send()
```

B/src/controllers/userController.js:55-57

```
55:   const { id } = req.params
56:   const user = await userService.updateUser(id, req.body)
57:   return res.status(200).json(user)
```

B/src/services/userService.js:78-92

```
78: async function updateUser(id, data) {
79:   const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data
80:
81:   if (!nome || !email || !senha || !tipo_login || !tipo_usuario) {
82:     const error = new Error('Todos os campos sao obrigatorios')
83:     error.statusCode = 400
```

```
84:     throw error
85:   }
86:
87:   const senhaCriptografada = passwordService.hashPassword(senha)
88:
89:   return await userRepository.update(
90:     id,
91:     { nome, email, senha: senhaCriptografada, tipo_login, tipo_usuario, id_medico, crm }
92:   )
```

B/src/repositories/userRepository.js:134-138

```
134:   await conn.execute(
135:     'UPDATE usuario SET nome = ?, email = ?, senha = ?, tipo_login = ?, tipo_usuario = ? WHERE
      id_usuario = ?',
136:     [nome, email, senha, tipo_login, tipo_usuario, id]
137:   )
138:   await conn.commit()
```

B/src/repositories/userRepository.js:18-30

```
18:   await conn.execute(
19:     'DELETE FROM paciente WHERE id_usuario = ?',
20:     [id]
21:   )
22:
23:   await conn.execute(
24:     'DELETE FROM medico WHERE id_usuario = ?',
25:     [id]
26:   )
27:
28:   await conn.execute(
29:     'DELETE FROM usuario WHERE id_usuario = ?',
30:     [id]
```

CORREÇÃO PROPOSTA

Autorizar leitura/edição/exclusão pela identidade autenticada; criar fluxo específico de troca de senha com reautenticação ou recuperação verificada. Excluir tipo_usuario/tipo_login da edição comum. Aplicar política explícita para alteração administrativa.

F04 — Registros, alarmes e preferências usam IDs e donos fornecidos pelo cliente

ALTA

PROBLEMA E EXPLORABILIDADE

GET/PUT/DELETE por ID verificam apenas existência. POST/PUT aceitam id_usuario ou id_registro sem confirmar posse/vínculo. As queries de objeto filtram apenas a chave primária; update de glicose também altera insulina e alarmes associados. As configurações são por usuário (idioma/tema/notificacoes), não um painel administrativo.

IMPACTO

Leitura, criação em nome de terceiros, alteração, transferência e exclusão de registros de saúde, lembretes, preferências e metadados de exportação. Pode adulterar histórico e desativar lembretes de outro usuário.

CONDIÇÕES E VALIDAÇÃO

API alcançável e IDs válidos; escrita exige os campos de cada service e relações aceitas pelo banco. Não exige login. Todos os handlers alcançáveis dessas famílias foram exercitados com banco simulado.

Evidências por arquivo e linha

B/src/repositories/registroGlicoseRepository.js:53-54

```
53:     WHERE rg.id_registro = ?,
54:     [id]
```

B/src/services/registroGlicoseService.js:125-130

```
125: function montarRegistroCompleto(data, registroAtual = {}) {
126:   const id_usuario = data.id_usuario ?? registroAtual.id_usuario
127:   const nivel_glicose = data.nivel_glicose ?? registroAtual.nivel_glicose
128:   const id_periodo = data.id_periodo ?? registroAtual.id_periodo
129:   const data_hora = data.data_hora ?? registroAtual.data_hora ?? formatarDataHoraAtual()
130:   const observacao = data.observacao ?? registroAtual.observacao ?? null
```

B/src/repositories/registroGlicoseRepository.js:198-205

```
198:     'UPDATE registroglicose SET id_usuario = ?, nivel_glicose = ?, data_hora = ?, id_periodo =
    ?, observacao = ? WHERE id_registro = ?',
199:     [
200:       glicose.id_usuario,
201:       glicose.nivel_glicose,
202:       glicose.data_hora,
203:       glicose.id_periodo,
204:       glicose.observacao ?? null,
205:       id
```

B/src/repositories/registroGlicoseRepository.js:278-289

```
278:     'DELETE FROM alarme WHERE id_registro = ?',
279:     [id]
280:   )
281:
282:   await conn.execute(
283:     'DELETE FROM registroinsulina WHERE id_registro = ?',
284:     [id]
285:   )
```

```
286:
287:     await conn.execute(
288:         'DELETE FROM registroglicose WHERE id_registro = ?',
289:         [id]
```

B/src/repositories/registroInsulinaRepository.js:14-15

```
14:     'SELECT id_registro_insulina, id_registro, id_tipo_insulina, unidade_insulina FROM registroin
    sulina WHERE id_registro_insulina = ?',
15:     [id]
```

B/src/repositories/registroInsulinaRepository.js:52-53

```
52:     'INSERT INTO registroinsulina (id_registro, id_tipo_insulina, unidade_insulina) VALUES (?,
    ?, ?)',
53:     [id_registro, id_tipo_insulina, unidade_insulina]
```

B/src/repositories/registroInsulinaRepository.js:80-81

```
80:     'UPDATE registroinsulina SET id_registro = ?, id_tipo_insulina = ?, unidade_insulina = ? WH
    ERE id_registro_insulina = ?',
81:     [id_registro, id_tipo_insulina, unidade_insulina, id]
```

B/src/repositories/registroInsulinaRepository.js:102-103

```
102:    'DELETE FROM registroinsulina WHERE id_registro_insulina = ?',
103:    [id]
```

B/src/repositories/alarmeRepository.js:32-33

```
32:     'SELECT id_alarme, id_usuario, data_hora, id_periodo, id_registro, dias_semana, ativo, tem_so
    m, tem_vibracao FROM alarme WHERE id_alarme = ?',
33:     [id]
```

B/src/repositories/alarmeRepository.js:47-48

```
47:     'INSERT INTO alarme (id_usuario, data_hora, id_periodo, id_registro, dias_semana, ativo, te
    m_som, tem_vibracao) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)',
48:     [id_usuario, data_hora, id_periodo || null, id_registro || null, dias_semana.join(','), ati
    vo, tem_som, tem_vibracao]
```

B/src/repositories/alarmeRepository.js:80-81

```
80:     'UPDATE alarme SET id_usuario = ?, data_hora = ?, id_periodo = ?, id_registro = ?, dias_sem
    ana = ?, ativo = ?, tem_som = ?, tem_vibracao = ? WHERE id_alarme = ?',
81:     [id_usuario, data_hora, id_periodo || null, id_registro || null, dias_semana.join(','), ati
    vo, tem_som, tem_vibracao, id]
```

B/src/repositories/alarmeRepository.js:107-108

```
107:    'DELETE FROM alarme WHERE id_alarme = ?',
108:    [id]
```

B/src/repositories/configuracaoRepository.js:14-15

```
14:     'SELECT id_configuracao, id_usuario, idioma, tema, notificacoes FROM configuracao WHERE id_co
    nfiguracao = ?',
15:     [id]
```

B/src/repositories/configuracaoRepository.js:29-30

```
29:     'INSERT INTO configuracao (id_usuario, idioma, tema, notificacoes) VALUES (?, ?, ?, ?)',
30:     [id_usuario, idioma, tema, notificacoes]
```

B/src/repositories/configuracaoRepository.js:58-59

```
58:      'UPDATE configuracao SET id_usuario = ?, idioma = ?, tema = ?, notificacoes = ? WHERE id_co  
nfiguracao = ?',  
59:      [id_usuario, idioma, tema, notificacoes, id]
```

B/src/repositories/configuracaoRepository.js:81-82

```
81:      'DELETE FROM configuracao WHERE id_configuracao = ?',  
82:      [id]
```

B/src/repositories/exportacaoRepository.js:14-15

```
14:      'SELECT id_exportacao, id_usuario, data, descricao FROM exportacao WHERE id_exportacao = ?',  
15:      [id]
```

B/src/repositories/exportacaoRepository.js:29-30

```
29:      'INSERT INTO exportacao (id_usuario, data, descricao) VALUES (?, ?, ?)',  
30:      [id_usuario, data, descricao]
```

B/src/repositories/exportacaoRepository.js:57-58

```
57:      'UPDATE exportacao SET id_usuario = ?, data = ?, descricao = ? WHERE id_exportacao = ?',  
58:      [id_usuario, data, descricao, id]
```

B/src/repositories/exportacaoRepository.js:79-80

```
79:      'DELETE FROM exportacao WHERE id_exportacao = ?',  
80:      [id]
```

CORREÇÃO PROPOSTA

Passar identidade autenticada aos services/repositories. Combinar ID do objeto e dono em SELECT/UPDATE/DELETE. Em insulina, validar o dono via registro/glicose; validar também id_registro dos alarmes. Derivar id_usuario no POST e impedir transferência pelo body. Validar relação médico-paciente quando aplicável, dentro da mesma transação.

F05 — Senha do MySQL permanece recuperável no histórico Git

MÉDIA

PROBLEMA E EXPLORABILIDADE

Duas versões do `.env` no histórico contêm `DB_PASSWORD` não vazio e `DB_USER=root`. O arquivo foi removido depois, mas os blobs continuam alcançáveis. O código carrega esse `.env` para configurar a conexão. O valor foi inspecionado localmente e mascarado em todos os artefatos da auditoria.

IMPACTO

Quem obtém o histórico consegue recuperar a senha antiga. Se ainda for válida ou reutilizada, permite acesso ao banco alcançável com os privilégios dessa credencial. Não há evidência de validade atual, de reutilização ou de exposição pública do repositório.

CONDIÇÕES E VALIDAÇÃO

Acesso ao histórico basta para extrair o segredo. Exploração do banco exige credencial ainda válida/reutilizada e acesso ao host. O `.env` histórico aponta localhost; não foi feita tentativa de autenticação. Severidade média pela exposição confirmada, sem pressupor compromisso atual.

Evidências por arquivo e linha

B/.env @ 147015e e 9ec3cab:3-4

```
3: DB_USER=root
4: DB_PASSWORD=[VALOR REAL MASCARADO]
```

Blobs d69f680ba9f1 e af0ed38a60e2; removido em fd64e41. Mesma família de exposição, uma issue.

B/src/config/database.js:4-13

```
4: require('dotenv').config({
5:   path: path.resolve(__dirname, '../.env')
6: })
7:
8: const pool = mysql.createPool({
9:   host: process.env.DB_HOST,
10:  port: Number(process.env.DB_PORT),
11:  user: process.env.DB_USER,
12:  password: process.env.DB_PASSWORD,
13:  database: process.env.DB_NAME,
```

CORREÇÃO PROPOSTA

Rotacionar a credencial e qualquer reutilização confirmada; usar usuário de banco com privilégios mínimos. Coordenar remoção dos blobs do histórico e clones, preservando evidências restritas. Manter apenas `.env.example` sem segredos e validar configuração de startup; usar scanner de segredos no CI.

F06 — Logs da API registram senhas e respostas sensíveis sem mascaramento

ALTA

PROBLEMA E EXPLORABILIDADE

O middleware registra req.body de toda chamada JSON, incluindo password no login e senha no cadastro/edição. Cadastro repete os dados no controller e no service antes do hash. Outro middleware registra corpos de resposta, incluindo dados de saúde e hashes da listagem. Não há condição de ambiente que desative esses logs.

IMPACTO

Leitores/coletadores de stdout obtêm credenciais em texto puro e dados pessoais/de saúde. Mesmo após corrigir a autenticação, o vazamento por logs permanece.

CONDIÇÕES E VALIDAÇÃO

Requer uma chamada com credenciais e acesso aos logs da aplicação/plataforma. Confirmado com senha fictícia; não foram acessados logs reais.

Evidências por arquivo e linha

B/src/app.js:39-48

```
39:   const retorno = Buffer.isBuffer(body)
40:     ? body.toString('utf8')
41:     : body
42:
43:   console.log('<retorno>', {
44:     metodo: req.method,
45:     rota: req.originalUrl,
46:     status: res.statusCode,
47:     body: retorno
48:   })
```

B/src/app.js:78-85

```
78: app.use((req, res, next) => {
79:   if (Object.keys(req.query).length > 0) {
80:     console.log('Query:', req.query)
81:   }
82:
83:   if (req.body && Object.keys(req.body).length > 0) {
84:     console.log('Body:', req.body)
85:   }
```

B/src/controllers/userController.js:43-47

```
43: async function create(req, res, next) {
44:   try {
45:     console.log('Request body:', req.body)
46:     const user = await userService.createUser(req.body)
47:     return res.status(201).json(user)
```

B/src/services/userService.js:37-40

```
37: async function createUser(data) {
38:   const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data
39:
40:   console.log('createUser:', data)
```

CORREÇÃO PROPOSTA

Remover logs integrais de body/resposta e duplicados de cadastro. Registrar somente metadados permitidos, status e identificador de correlação. Mascara recursivamente campos sensíveis antes de qualquer logging e revisar retenção/acesso aos logs já existentes.

F07 — App persiste a senha reutilizável em SharedPreferences

MÉDIA

PROBLEMA E EXPLORABILIDADE

Após login, a senha original é gravada automaticamente em `saved_password` com `prefs.setString` e lida para login automático. Não há criptografia na camada do aplicativo nem armazenamento de sessão revogável nesse fluxo.

IMPACTO

Leitura das preferências do aplicativo/perfil permite recuperar a senha original e reutilizá-la. Não é chave hardcoded nem XSS; é um achado adicional de armazenamento de credenciais.

CONDIÇÕES E VALIDAÇÃO

Requer acesso às preferências/perfil ou extração de dados do app; não se assume que outro app sem privilégios possa ler o sandbox Android. O fluxo ocorre após login bem-sucedido.

Evidências por arquivo e linha

F/lib/states/login_form_state.dart:54-59

```
54:     final loginData = await AuthService().login(username, password);
55:     await _savedLoginService.saveCredentials(
56:       userId: loginData.userId,
57:       username: username,
58:       password: password,
59:     );
```

F/lib/services/local/saved_login_service.dart:25-28

```
25:     final prefs = await SharedPreferences.getInstance();
26:     await prefs.setInt(_userIdKey, userId);
27:     await prefs.setString(_usernameKey, username);
28:     await prefs.setString(_passwordKey, password);
```

F/lib/services/local/saved_login_service.dart:31-35

```
31: Future<SavedLoginData?> getCredentials() async {
32:   final prefs = await SharedPreferences.getInstance();
33:   final userId = prefs.getInt(_userIdKey);
34:   final username = prefs.getString(_usernameKey);
35:   final password = prefs.getString(_passwordKey);
```

F/lib/states/login_form_state.dart:111-115

```
111:   try {
112:     final loginData = await AuthService().login(
113:       savedCredentials.username,
114:       savedCredentials.password,
115:     );
```

CORREÇÃO PROPOSTA

Eliminar persistência de senha e apagar `saved_password` em migração. Após implementar sessão no servidor, usar credencial revogável protegida pelo mecanismo seguro da plataforma; no web, projetar sessão apropriada sem senha no armazenamento do navegador.

F08 — Login e dados de saúde trafegam em HTTP sem TLS

ALTA

PROBLEMA E EXPLORABILIDADE

ApiIpService monta sempre `http://<ip>:3000`. ApiService envia as credenciais no corpo JSON a essa URL; exportação usa a mesma base. O manifesto Android permite cleartext no aplicativo principal, sem limitar ao debug. O servidor inicia HTTP em 0.0.0.0.

IMPACTO

Um observador no caminho de rede lê senhas e dados de saúde; um intermediário ativo pode adulterar respostas e capturar credenciais. Não foi executada interceptação de tráfego.

CONDIÇÕES E VALIDAÇÃO

Cliente Android/ambiente que permita HTTP, conexão direta conforme configuração e atacante capaz de observar/interferir no caminho. Plataformas que bloqueiem cleartext podem falhar em conectar. Proteções externas como VPN/gateway não foram inspecionadas.

Evidências por arquivo e linha

F/lib/services/local/api_ip_service.dart:18-26

```
18: Future<String> getBaseUrl() async {
19:   final savedIp = await getApiIpDigits();
20:   final ip =
21:     (isValidIp(savedIp) ? savedIp : null) ??
22:     formatDigitsAsIp(savedIp) ??
23:     formatDigitsAsIp(_defaultApiIpDigits) ??
24:     '10.173.57.47';
25:
26:   return 'http://$ip:$_apiPort';
```

F/lib/services/api/auth_service.dart:14-17

```
14:   final response = await _apiService.post('login', {
15:     'username': username,
16:     'password': password,
17:   });
```

F/lib/services/api/api_service.dart:30-32

```
30:   final response = await http
31:     .post(uri, headers: headers, body: jsonEncode(body))
32:     .timeout(const Duration(seconds: 10));
```

F/android/app/src/main/AndroidManifest.xml:13-15

```
13:   android:icon="@mipmap/ic_launcher"
14:   android:enableOnBackInvokedCallback="true"
15:   android:usesCleartextTraffic="true">
```

B/src/server.js:13-16

```
13:   await testDatabaseConnection();
14:
15:   app.listen(PORT, '0.0.0.0', () => {
16:     console.log(`Servidor rodando em http://${getLocalIP()}:${PORT}`);
```

CORREÇÃO PROPOSTA

Configurar endpoint HTTPS validado e rejeitar esquema HTTP em release. Desabilitar `usesCleartextTraffic` na configuração de produção; eventual exceção local deve existir somente em debug. Manter validação normal de certificado e configurar TLS na terminação publicada.

Cobertura de todos os handlers

48 registros em 9 routers, mais GET / no app. 47 handlers dos routers exercitados por HTTP local, com app/controllers/services/repositories reais e apenas pool/conexão de banco simulados. Todos aceitaram payloads válidos sem cookie/token. O login recebeu senha fictícia válida; sua exposição pública é esperada. Catálogos de leitura não são tratados como dados privados.

Um handler registrado não é alcançável: GET /usuarios/:tipo_usuario é capturado primeiro por GET /usuarios/:id (userRoutes.js:9-10). Seu código foi revisado, mas não é reportado como caminho adicional explorável. GET / e OPTIONS foram revisados estaticamente.

B/src/routes/authRoutes.js

Cadeia revisada: src/controllers/authController.js → src/services/authService.js → src/services/passwordService.js → src/repositories/userRepository.js

Rota / linha	Handler / HTTP	Resultado
POST /login Linha 6	authController.login 200	Público: autentica senha; sem sessão

B/src/routes/userRoutes.js

Cadeia revisada: src/controllers/userController.js → src/services/userService.js → src/repositories/userRepository.js

Rota / linha	Handler / HTTP	Resultado
GET /usuarios/ Linha 8	userController.index 200	F01
GET /usuarios/:id Linha 9	userController.show 200	F03
GET /usuarios/:tipo_usuario Linha 10	showByType Sombreado	Sombreado; service não valida permissão
POST /usuarios/ Linha 12	userController.create 201	F03
DELETE /usuarios/:id Linha 13	userController.deleteById 204	F03
PUT /usuarios/:id Linha 14	userController.update 200	F03

B/src/routes/typeInsuRoutes.js

Cadeia revisada: src/controllers/typeInsuController.js → src/services/typeInsuService.js → src/repositories/typeInsuRepository.js

Rota / linha	Handler / HTTP	Resultado
--------------	----------------	-----------

Rota / linha	Handler / HTTP	Resultado
GET /tipos-insulina/ Linha 6	typeInsuController.index 200	Leitura de catálogo; não se presume privada
GET /tipos-insulina/:id Linha 7	typeInsuController.show 200	Leitura de catálogo; não se presume privada
POST /tipos-insulina/ Linha 8	typeInsuController.create 201	F02
PUT /tipos-insulina/:id Linha 9	typeInsuController.update 200	F02
DELETE /tipos-insulina/:id Linha 10	typeInsuController.deleteById 204	F02

B/src/routes/periodoRoutes.js

Cadeia revisada: src/controllers/periodoController.js → src/services/periodoService.js → src/repositories/periodoRepository.js

Rota / linha	Handler / HTTP	Resultado
GET /periodos/ Linha 6	periodoController.index 200	Leitura de catálogo; não se presume privada
GET /periodos/:id Linha 7	periodoController.show 200	Leitura de catálogo; não se presume privada
POST /periodos/ Linha 8	periodoController.create 201	F02
PUT /periodos/:id Linha 9	periodoController.update 200	F02
DELETE /periodos/:id Linha 10	periodoController.deleteById 204	F02

B/src/routes/registroGlicoseRoutes.js

Cadeia revisada: src/controllers/registroGlicoseController.js → src/services/registroGlicoseService.js → src/repositories/registroGlicoseRepository.js

Rota / linha	Handler / HTTP	Resultado
GET /registros-glicose/ Linha 6	registroGlicoseController.index 200	F01
GET /registros-glicose/dashboard Linha 7	registroGlicoseController.getDashboard 200	F01

Rota / linha	Handler / HTTP	Resultado
GET /registros-glicose/usuario/:id_usuario/historico Linha 8	registroGlicoseController.getHistorico 200	F01
GET /registros-glicose/usuario/:id_usuario Linha 9	registroGlicoseController.showByUserId 200	F01
GET /registros-glicose/:id Linha 10	registroGlicoseController.show 200	F04
POST /registros-glicose/ Linha 11	registroGlicoseController.create 201	F04
PUT /registros-glicose/:id Linha 12	registroGlicoseController.update 200	F04
DELETE /registros-glicose/:id Linha 13	registroGlicoseController.deleteById 204	F04

B/src/routes/registroInsulinaRoutes.js

Cadeia revisada: src/controllers/registroInsulinaController.js → src/services/registroInsulinaService.js → src/repositories/registroInsulinaRepository.js

Rota / linha	Handler / HTTP	Resultado
GET /registros-insulina/ Linha 6	registroInsulinaController.index 200	F01
GET /registros-insulina/usuario/:id_usuario Linha 7	registroInsulinaController.showByUserId 200	F01
GET /registros-insulina/:id Linha 8	registroInsulinaController.show 200	F04
POST /registros-insulina/ Linha 9	registroInsulinaController.create 201	F04
PUT /registros-insulina/:id Linha 10	registroInsulinaController.update 200	F04
DELETE /registros-insulina/:id Linha 11	registroInsulinaController.deleteById 204	F04

B/src/routes/exportacaoRoutes.js

Cadeia revisada: src/controllers/exportacaoController.js → src/services/exportacaoService.js → src/repositories/exportacaoRepository.js

Rota / linha	Handler / HTTP	Resultado
GET /exportacoes/ Linha 6	exportacaoController.index 200	F01
GET /exportacoes/relatorio Linha 7	exportacaoController.gerarRelat orio 200	F01
GET /exportacoes/:id Linha 8	exportacaoController.show 200	F04
POST /exportacoes/ Linha 9	exportacaoController.create 201	F04
PUT /exportacoes/:id Linha 10	exportacaoController.update 200	F04
DELETE /exportacoes/:id Linha 11	exportacaoController.deleteByld 204	F04

B/src/routes/configuracaoRoutes.js

Cadeia revisada: src/controllers/configuracaoController.js → src/services/configuracaoService.js → src/repositories/configuracaoRepository.js

Rota / linha	Handler / HTTP	Resultado
GET /configuracoes/ Linha 6	configuracaoController.index 200	F01
GET /configuracoes/:id Linha 7	configuracaoController.show 200	F04
POST /configuracoes/ Linha 8	configuracaoController.create 201	F04
PUT /configuracoes/:id Linha 9	configuracaoController.update 200	F04
DELETE /configuracoes/:id Linha 10	configuracaoController.deleteByld 204	F04

B/src/routes/alarmeRoutes.js

Cadeia revisada: src/controllers/alarmeController.js → src/services/alarmeService.js → src/repositories/alarmeRepository.js

Rota / linha	Handler / HTTP	Resultado
GET /alarmes/ Linha 6	alarmeController.index 200	F01
GET /alarmes/usuario/:usuariold Linha 7	alarmeController.showByUsuario Id 200	F01
GET /alarmes/:id Linha 8	alarmeController.show 200	F04
POST /alarmes/ Linha 9	alarmeController.create 201	F04
PUT /alarmes/:id Linha 10	alarmeController.update 200	F04
DELETE /alarmes/:id Linha 11	alarmeController.deleteById 204	F04

GET /: B/src/app.js:90–94 retorna mensagem fixa. OPTIONS: B/src/app.js:64–74 responde 204; CORS limita leitura no navegador, mas não autentica clientes HTTP. B/src/middlewares/errorHandler.js:1–27 também foi revisado.

Cruzamento frontend ↔ API

Não foram encontrados gates isAdmin/canEdit/role nem branches de UI por tipo_usuario. A única atribuição de tipo_usuario no app é paciente no cadastro. A navegação após login e IDs em Globals são estado local; não substituem autorização.

Login/cadastro

lib/services/api/auth_service.dart:14-17,89-96

POST /login; POST /usuarios

Sem gate de papel; paciente é valor fixo. Senha conferida no login, cadastro público. F06/F08; não prova escalada de papel.

Histórico e resumo

lib/services/api/data_service.dart:26-31,58-66,170-173

GET /registros-glicose/usuario/:id; /historico; GET /registros-insulina/usuario/:id

ID do estado local; servidor não confirma identidade/posse. F01.

Alarmes

lib/services/api/data_service.dart:87,107,120-123,147,157

GET /alarmes/usuario/:id; POST /alarmes; PUT/DELETE /alarmes/:id

Sem gate de papel; servidor confere campos/existência. F01/F04.

Glicose

lib/services/api/data_service.dart:196,212,223,242

POST /registros-glicose; GET/PUT/DELETE /registros-glicose/:id

Sem gate de papel; objeto selecionado na UI não autentica o dono. F04.

Exportação

lib/services/api/report_export_service.dart:51-70

GET /exportacoes/relatorio

Valida ID/período no cliente e servidor; nenhuma autorização. F01.

Demais rotas

src/routes/{user,periodo,typeInsu,configuracao,exportacao}Routes.js

Gestão de usuários, catálogos, preferências e metadados

Sem UI correspondente de gestão de papel; todas as rotas foram revisadas mesmo sem chamada do app. F01-F04.

Histórico, bundle e verificações

Segredos: o histórico contém duas versões de .env com senha de banco não vazia (F05). Três candidatos em database.js eram referências process.env, não segredos literais. O HEAD lê ambiente e não traz defaults de senha; .gitignore do backend ignora .env, mas isso não remove commits antigos.

Bundle: flutter build web --release --no-pub concluiu. Foram inventariados 40 arquivos em build/web; main.dart.js tem 3.271.579 bytes. Busca por tokens conhecidos e atribuições de segredos não encontrou candidatos; busca exata pelas credenciais históricas também não encontrou correspondência. Isso cobre o bundle local gerado, não um artefato publicado em loja/servidor.

XSS: busca por innerHTML/outerHTML, equivalentes, WebView/HtmlWidget, eval/new Function, javascript:, URLs de usuário e renderização markdown/HTML; revisão dos pontos de texto e saída. Nenhum fluxo confirmado. web/index.html usa recursos estáticos. O SVG do relatório vem de asset local fixo. Não há sanitizador HTML aplicado pela aplicação; no fluxo atual não existe sink que demande essa sanitização.

Validação: 21 testes existentes da API passaram. O script verificar-api.cjs confirmou 47 handlers alcançáveis, atualização anônima de senha, hash em GET /usuarios e senha fictícia nos logs. Não foram executadas queries reais. Testes não provam comportamento das constraints ou segurança de um deploy externo.

A compilação web emitiu aviso de referência a CupertinoIcons sem fonte correspondente, mas concluiu. Isso não é achado de segurança. Compilação e testes locais precisaram de execução fora do sandbox para cache Flutter e socket loopback.

Reprodução e rastreabilidade: README.md explica os comandos; inventario-evidencias.json registra hashes, contagens e candidatos sem valores secretos; verificacao-api.json lista resultado por rota. Os scripts de relatório usam o snapshot revisado, não substituem nova auditoria após alterações no código.

Recomendações priorizadas

Prioridade	Ação / aceite principal
P1 — F03, F01, F04	Implementar identidade validada e autorização por dono em toda rota privada; bloquear troca de senha alheia, listagens globais e associação de registros entre pacientes. Testar anônimo, A→A, A→B e vínculo médico-paciente.
P1 — F02	Restringir manutenção de catálogos a permissão explícita ou remover endpoints de escrita não usados.
P1 — F06, F08	Eliminar corpos sensíveis dos logs e usar HTTPS no cliente release. Confirmar que senha sintética não aparece em stdout ou tráfego legível.
P2 — F05	Rotacionar credencial histórica, verificar reutilização e coordenar limpeza do Git; não basta apagar o arquivo no HEAD.
P2 — F07	Migrar senha persistida para sessão revogável e apagar saved_password; depende do mecanismo de sessão de P1.
P3 — sustentação	Adicionar testes de autorização por endpoint, scanner de segredos no CI e revisão de configuração de startup. Manter projeção de resposta sem senha, validações de exportação e consultas parametrizadas.

Validar permissões a cada requisição e negar acesso por padrão orienta P1. Rotação e gestão do ciclo de vida dos segredos orientam F05. Referências de remediação da OWASP; os achados se baseiam exclusivamente no código local.

OWASP Authorization Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

OWASP Secrets Management Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

ISSUES PARA O GITHUB

Textos completos em Markdown, prontos para copiar. Também disponíveis em issues-github.md. Nenhuma issue foi publicada. Evidências relacionadas foram agrupadas por causa e correção para evitar duplicação.

```

--- ISSUE 1 ---
# [Segurança] Leitura global de usuários, dados de saúde e relatórios sem identidade validada
Labels sugeridas: security, alta
Achado: F01
## Problema
O login apenas devolve os dados do usuário. Nenhum middleware estabelece identidade autenticada nas rotas. As listagens retornam todos os donos; GET /usuarios também devolve a coluna senha por SELECT *. Nas buscas por usuário, dashboard, histórico e exportação, o filtro id_usuario usa diretamente path/query do chamador. É possível consultar outro paciente, inclusive por nome/e-mail nas listagens de glicose/insulina. Não existe RLS ou middleware de tenant: há filtros manuais de dono, sem vínculo com autenticação.
## Condições de explorabilidade
Acesso de rede à API e dados existentes. Não exige login, flag ou configuração adicional no código. Não foi inspecionado um eventual gateway externo. Teste local confirmou resposta 200 e campo senha com hash; banco simulado.
## Evidência
backend/src/app.js:96-104
...

96: app.use('/', authRoutes)
97: app.use('/usuarios', userRoutes)
98: app.use('/tipos-insulina', typeInsuRoutes)
99: app.use('/periodos', periodoRoutes)
100: app.use('/registros-glicose', registroGlicoseRoutes)
101: app.use('/registros-insulina', registroInsulinaRoutes)
102: app.use('/exportacoes', exportacaoRoutes)
103: app.use('/configuracoes', configuracaoRoutes)
104: app.use('/alarmes', alarmeRoutes)
...

backend/src/repositories/userRepository.js:4-9
...

4: async function findAll() {
5:   const [rows] = await db.execute(
6:     'SELECT * FROM usuario ORDER BY id_usuario ASC'
7:   )
8:
9:   return rows
...

backend/src/controllers/exportacaoController.js:54-60
...

54:   const { id_usuario, dataInicio, dataFim, formato } = req.query
55:   const arquivo = await exportacaoService.gerarRelatorio({
56:     idUsuario: id_usuario,
57:     dataInicio,
58:     dataFim,
59:     formato
60:   })
...

backend/src/repositories/exportacaoRepository.js:108-111
...

108:   WHERE rg.id_usuario = ? AND rg.data_hora >= ?
109:     AND rg.data_hora < DATE_ADD(?, INTERVAL 1 DAY)
110:   ORDER BY rg.data_hora ASC, ri.id_registro_insulina ASC`,
111:   [idUsuario, dataInicio, dataFim]

```

```
```
```

## ## Impacto

Exposição de nomes, e-mails, hashes de senha, medições, doses, alarmes, preferências e relatórios de outros pacientes. Se existirem senhas legadas em texto puro, também saem na listagem; a existência desses registros não foi comprovada.

## ## Sugestão de correção

Criar sessão/token validado no servidor e negar acesso por padrão. Derivar o dono da identidade e, para médicos, validar o vínculo autorizado com o paciente. Aplicar o mesmo escopo a listagens, dashboard, histórico e exportação. Substituir SELECT \* por projeção sem senha. Remover listagens globais desnecessárias.

## ## Critérios de aceite

- [ ] Requisições anônimas às rotas privadas retornam 401.
- [ ] Paciente A não lê dados, agregados nem relatórios de B por ID, nome ou e-mail.
- [ ] Médico sem vínculo com B recebe 403/404; vínculo válido funciona.
- [ ] Nenhuma resposta pública inclui senha ou hash; teste cobre GET /usuarios.

```
--- FIM ISSUE 1 ---
```

# ISSUES PARA O GITHUB — 2/8

--- ISSUE 2 ---

# [Segurança] Catálogos globais aceitam criação, edição e exclusão anônimas

Labels sugeridas: security, alta

Achado: F02

## Problema

POST, PUT e DELETE de /periodos e /tipos-insulina chamam diretamente os controllers. Os services verificam campos/existência, mas nenhum privilégio. As tabelas são catálogos compartilhados, sem coluna de dono. Não há isAdmin/canEdit/role nem UI de manutenção desses catálogos no Flutter; portanto não se afirma um gate de papel escondido no navegador. A falha verificada é a ausência de autorização no servidor para escrita global.

## Condições de explorabilidade

API alcançável. Edição exige ID existente e campo nome/descricao; exclusão pode ser impedida por foreign keys se o catálogo estiver em uso. A edição anônima foi exercitada com banco simulado.

## Evidência

backend/src/routes/periodoRoutes.js:8-10

...

8: router.post('/', periodoController.create)

9: router.put('/:id', periodoController.update)

10: router.delete('/:id', periodoController.deleteById)

...

backend/src/routes/typeInsuRoutes.js:8-10

...

8: router.post('/', typeInsuController.create)

9: router.put('/:id', typeInsuController.update)

10: router.delete('/:id', typeInsuController.deleteById)

...

backend/src/repositories/periodoRepository.js:55-56

...

55: 'UPDATE periodo SET descricao = ? WHERE id\_periodo = ?',

56: [descricao, id]

...

backend/src/repositories/typeInsuRepository.js:37-38

...

37: 'UPDATE tipoinsulina SET nome = ? WHERE id\_tipo\_insulina = ?',

38: [nome, id]

...

## Impacto

Qualquer chamador altera nomes/tipos/periódos usados por todos os usuários, cria entradas arbitrárias e remove entradas não referenciadas.

## Sugestão de correção

Exigir identidade e permissão explícita de manutenção dos catálogos em todas as três operações. Se não houver função administrativa no produto, remover/desabilitar as rotas de escrita e manter carga controlada de catálogos.

## Critérios de aceite

- [ ] POST/PUT/DELETE dos dois catálogos rejeitam anônimos e usuários sem permissão.
- [ ] Usuário autorizado mantém o catálogo; leitura segue a política definida.
- [ ] Autorização ocorre antes da consulta/mutação no banco.

--- FIM ISSUE 2 ---

## ISSUES PARA O GITHUB — 3/8

--- ISSUE 3 ---

# [Segurança] Atualização de conta por ID permite trocar a senha de outro usuário

Labels sugeridas: security, crítica

Achado: F03

## Problema

PUT /usuarios/:id aceita nome, email, senha, tipo\_login e tipo\_usuario do body, calcula o hash e executa UPDATE usuario WHERE id\_usuario = ?. Não verifica sessão, posse, senha atual nem permissão sobre o papel. GET e DELETE da conta também usam somente o ID. Conhecendo um ID e os campos retornados pelas listagens, o chamador pode substituir a senha da vítima.

## Condições de explorabilidade

API alcançável, usuário existente e body com os campos obrigatórios. Restrições reais do banco podem impedir valores inválidos ou exclusão com referências. Teste com services/repository reais e banco simulado confirmou hash da nova senha e ID escolhido.

## Evidência

backend/src/routes/userRoutes.js:8-14

...

```
8: router.get('/', userController.index);
9: router.get('/:id', userController.show);
10: router.get('/:tipo_usuario', userController.showByType);
11:
12: router.post('/', userController.create);
13: router.delete('/:id', userController.deleteById);
14: router.put('/:id', userController.update);
...
```

backend/src/services/userService.js:78-92

...

```
78: async function updateUser(id, data) {
79: const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data
80:
81: if (!nome || !email || !senha || !tipo_login || !tipo_usuario) {
82: const error = new Error('Todos os campos sao obrigatorios')
83: error.statusCode = 400
84: throw error
85: }
86:
87: const senhaCriptografada = passwordService.hashPassword(senha)
88:
89: return await userRepository.update(
90: id,
91: { nome, email, senha: senhaCriptografada, tipo_login, tipo_usuario, id_medico, crm }
92:)
...
```

backend/src/repositories/userRepository.js:134-138

...

```
134: await conn.execute(
135: 'UPDATE usuario SET nome = ?, email = ?, senha = ?, tipo_login = ?, tipo_usuario = ? WHERE
 id_usuario = ?',
136: [nome, email, senha, tipo_login, tipo_usuario, id]
137:)
138: await conn.commit()
...
```

## Impacto

Tomada de conta, alteração de identidade/papel e exclusão de contas. A alteração de papel é persistida, mas não foi demonstrado um privilégio adicional de médico, pois o projeto não implementa esse controle.

## Sugestão de correção

Autorizar leitura/edição/exclusão pela identidade autenticada; criar fluxo específico de troca de sen

ha com reautenticação ou recuperação verificada. Excluir tipo\_usuario/tipo\_login da edição comum. Aplicar política explícita para alteração administrativa.

## Critérios de aceite

- [ ] A não consegue ler/alterar/excluir B mudando :id.
  - [ ] Anônimo recebe 401 antes de hash/UPDATE.
  - [ ] Senha exige reautenticação/recuperação válida.
  - [ ] Campos de papel não podem ser modificados pela atualização comum.
- FIM ISSUE 3 ---

# ISSUES PARA O GITHUB — 4/8

--- ISSUE 4 ---

# [Segurança] Registros, alarmes e preferências usam IDs e donos fornecidos pelo cliente

Labels sugeridas: security, alta

Achado: F04

## Problema

GET/PUT/DELETE por ID verificam apenas existência. POST/PUT aceitam id\_usuario ou id\_registro sem confirmar posse/vínculo. As queries de objeto filtram apenas a chave primária; update de glicose também altera insulina e alarmes associados. As configurações são por usuário (idioma/tema/notificacoes), não um painel administrativo.

## Condições de explorabilidade

API alcançável e IDs válidos; escrita exige os campos de cada service e relações aceitas pelo banco.

Não exige login. Todos os handlers alcançáveis dessas famílias foram exercitados com banco simulado.

## Evidência

backend/src/services/registroGlicoseService.js:125–130

...

```
125: function montarRegistroCompleto(data, registroAtual = {}) {
126: const id_usuario = data.id_usuario ?? registroAtual.id_usuario
127: const nivel_glicose = data.nivel_glicose ?? registroAtual.nivel_glicose
128: const id_periodo = data.id_periodo ?? registroAtual.id_periodo
129: const data_hora = data.data_hora ?? registroAtual.data_hora ?? formatarDataHoraAtual()
130: const observacao = data.observacao ?? registroAtual.observacao ?? null
...
```

backend/src/repositories/registroGlicoseRepository.js:198–205

...

```
198: 'UPDATE registroglicose SET id_usuario = ?, nivel_glicose = ?, data_hora = ?, id_periodo =
 ?, observacao = ? WHERE id_registro = ?',
199: [
200: glicose.id_usuario,
201: glicose.nivel_glicose,
202: glicose.data_hora,
203: glicose.id_periodo,
204: glicose.observacao ?? null,
205: id
...
```

backend/src/repositories/registroInsulinaRepository.js:80–81

...

```
80: 'UPDATE registroinsulina SET id_registro = ?, id_tipo_insulina = ?, unidade_insulina = ? WH
 ERE id_registro_insulina = ?',
81: [id_registro, id_tipo_insulina, unidade_insulina, id]
...
```

backend/src/repositories/alarmeRepository.js:80–81

...

```
80: 'UPDATE alarme SET id_usuario = ?, data_hora = ?, id_periodo = ?, id_registro = ?, dias_sem
 ana = ?, ativo = ?, tem_som = ?, tem_vibracao = ? WHERE id_alarme = ?',
81: [id_usuario, data_hora, id_periodo || null, id_registro || null, dias_semana.join(','), ati
 vo, tem_som, tem_vibracao, id]
...
```

backend/src/repositories/configuracaoRepository.js:58–59

...

```
58: 'UPDATE configuracao SET id_usuario = ?, idioma = ?, tema = ?, notificacoes = ? WHERE id_co
 nfiguracao = ?',
59: [id_usuario, idioma, tema, notificacoes, id]
...
```

backend/src/repositories/exportacaoRepository.js:57–58

...

```
57: 'UPDATE exportacao SET id_usuario = ?, data = ?, descricao = ? WHERE id_exportacao = ?',
```

```
58: [id_usuario, data, descricao, id]
 ...
```

#### ## Impacto

Leitura, criação em nome de terceiros, alteração, transferência e exclusão de registros de saúde, lembretes, preferências e metadados de exportação. Pode adulterar histórico e desativar lembretes de outro usuário.

#### ## Sugestão de correção

Passar identidade autenticada aos services/repositories. Combinar ID do objeto e dono em SELECT/UPDATE/DELETE. Em insulina, validar o dono via registroglicose; validar também id\_registro dos alarmes. Derivar id\_usuario no POST e impedir transferência pelo body. Validar relação médico-paciente quando aplicável, dentro da mesma transação.

#### ## Critérios de aceite

- [ ] A não acessa nem altera objetos de B por path/body/query.
  - [ ] POST com id\_usuario/id\_registro de B é rejeitado ou substituído por dono autenticado conforme contrato.
  - [ ] UPDATE não transfere dono nem associa registro de outro paciente.
  - [ ] Exclusão/edição completa de glicose protege também alarmes e insulina.
- FIM ISSUE 4 ---

# ISSUES PARA O GITHUB — 5/8

--- ISSUE 5 ---

# [Segurança] Senha do MySQL permanece recuperável no histórico Git

Labels sugeridas: security, média

Achado: F05

## Problema

Duas versões do .env no histórico contêm DB\_PASSWORD não vazio e DB\_USER=root. O arquivo foi removido depois, mas os blobs continuam alcançáveis. O código carrega esse .env para configurar a conexão. O valor foi inspecionado localmente e mascarado em todos os artefatos da auditoria.

## Condições de explorabilidade

Acesso ao histórico basta para extrair o segredo. Exploração do banco exige credencial ainda válida/reutilizada e acesso ao host. O .env histórico aponta localhost; não foi feita tentativa de autenticação. Severidade média pela exposição confirmada, sem pressupor compromisso atual.

## Evidência

backend/.env @ 147015e e 9ec3cab:3-4

...

3: DB\_USER=root

4: DB\_PASSWORD=[VALOR REAL MASCARADO]

...

backend/src/config/database.js:4-13

...

4: require('dotenv').config({

5: path: path.resolve(\_\_dirname, '../.env')

6: })

7:

8: const pool = mysql.createPool({

9: host: process.env.DB\_HOST,

10: port: Number(process.env.DB\_PORT),

11: user: process.env.DB\_USER,

12: password: process.env.DB\_PASSWORD,

13: database: process.env.DB\_NAME,

...

## Impacto

Quem obtém o histórico consegue recuperar a senha antiga. Se ainda for válida ou reutilizada, permite acesso ao banco alcançável com os privilégios dessa credencial. Não há evidência de validade atual, de reutilização ou de exposição pública do repositório.

## Sugestão de correção

Rotacionar a credencial e qualquer reutilização confirmada; usar usuário de banco com privilégios mínimos. Coordenar remoção dos blobs do histórico e clones, preservando evidências restritas. Manter apenas .env.example sem segredos e validar configuração de startup; usar scanner de segredos no CI.

## Critérios de aceite

- [ ] Credencial antiga revogada ou sua revogação comprovada sem registrar o valor na issue.
- [ ] Segredo não está em referências distribuídas após limpeza coordenada.
- [ ] CI rejeita commit de .env/segredo real.
- [ ] Aplicação recebe credenciais externamente e rejeita configuração inválida.

--- FIM ISSUE 5 ---

# ISSUES PARA O GITHUB — 6/8

--- ISSUE 6 ---

# [Segurança] Logs da API registram senhas e respostas sensíveis sem mascaramento

Labels sugeridas: security, alta

Achado: F06

## Problema

O middleware registra req.body de toda chamada JSON, incluindo password no login e senha no cadastro/edição. Cadastro repete os dados no controller e no service antes do hash. Outro middleware registra corpos de resposta, incluindo dados de saúde e hashes da listagem. Não há condição de ambiente que desative esses logs.

## Condições de explorabilidade

Requer uma chamada com credenciais e acesso aos logs da aplicação/plataforma. Confirmado com senha fictícia; não foram acessados logs reais.

## Evidência

backend/src/app.js:78–85

...

```
78: app.use((req, res, next) => {
79: if (Object.keys(req.query).length > 0) {
80: console.log('Query:', req.query)
81: }
82:
83: if (req.body && Object.keys(req.body).length > 0) {
84: console.log('Body:', req.body)
85: }
...

```

backend/src/controllers/userController.js:43–47

...

```
43: async function create(req, res, next) {
44: try {
45: console.log('Request body:', req.body)
46: const user = await userService.createUser(req.body)
47: return res.status(201).json(user)
...

```

backend/src/services/userService.js:37–40

...

```
37: async function createUser(data) {
38: const { nome, email, senha, tipo_login, tipo_usuario, id_medico, crm } = data
39:
40: console.log('createUser:', data)
...

```

## Impacto

Leitores/coletadores de stdout obtêm credenciais em texto puro e dados pessoais/de saúde. Mesmo após corrigir a autenticação, o vazamento por logs permanece.

## Sugestão de correção

Remover logs integrais de body/resposta e duplicados de cadastro. Registrar somente metadados permitidos, status e identificador de correlação. Mascaram recursivamente campos sensíveis antes de qualquer logging e revisar retenção/acesso aos logs já existentes.

## Critérios de aceite

- [ ] Login, cadastro e edição não registram senha/password/hash/token em sucesso ou erro.
- [ ] Dados de saúde e corpos completos não são logados.
- [ ] Teste captura console/logger e verifica ausência de credenciais sintéticas.

--- FIM ISSUE 6 ---

# ISSUES PARA O GITHUB — 7/8

--- ISSUE 7 ---

# [Segurança] App persiste a senha reutilizável em SharedPreferences

Labels sugeridas: security, média

Achado: F07

## Problema

Após login, a senha original é gravada automaticamente em saved\_password com prefs.setString e lida para login automático. Não há criptografia na camada do aplicativo nem armazenamento de sessão revogável nesse fluxo.

## Condições de explorabilidade

Requer acesso às preferências/perfil ou extração de dados do app; não se assume que outro app sem privilégios possa ler o sandbox Android. O fluxo ocorre após login bem-sucedido.

## Evidência

frontend/lib/states/login\_form\_state.dart:54–59

...

```
54: final loginData = await AuthService().login(username, password);
55: await _savedLoginService.saveCredentials(
56: userId: loginData.userId,
57: username: username,
58: password: password,
59:);
```

...

frontend/lib/services/local/saved\_login\_service.dart:25–28

...

```
25: final prefs = await SharedPreferences.getInstance();
26: await prefs.setInt(_userIdKey, userId);
27: await prefs.setString(_usernameKey, username);
28: await prefs.setString(_passwordKey, password);
```

...

frontend/lib/services/local/saved\_login\_service.dart:31–35

...

```
31: Future<SavedLoginData?> getCredentials() async {
32: final prefs = await SharedPreferences.getInstance();
33: final userId = prefs.getInt(_userIdKey);
34: final username = prefs.getString(_usernameKey);
35: final password = prefs.getString(_passwordKey);
```

...

## Impacto

Leitura das preferências do aplicativo/perfil permite recuperar a senha original e reutilizá-la. Não é chave hardcoded nem XSS; é um achado adicional de armazenamento de credenciais.

## Sugestão de correção

Eliminar persistência de senha e apagar saved\_password em migração. Após implementar sessão no servidor, usar credencial revogável protegida pelo mecanismo seguro da plataforma; no web, projetar sessão apropriada sem senha no armazenamento do navegador.

## Critérios de aceite

- [ ] Senha não é gravada nas preferências após login nem auto-login.
- [ ] Migração remove saved\_password de instalações existentes.
- [ ] Logout revoga/limpa sessão; auto-login usa sessão válida, não senha persistida.

--- FIM ISSUE 7 ---

# ISSUES PARA O GITHUB — 8/8

--- ISSUE 8 ---

# [Segurança] Login e dados de saúde trafegam em HTTP sem TLS

Labels sugeridas: security, alta

Achado: F08

## Problema

ApiIpService monta sempre `http://<ip>:3000`. ApiService envia as credenciais no corpo JSON a essa URL; exportação usa a mesma base. O manifesto Android permite cleartext no aplicativo principal, sem limitar ao debug. O servidor inicia HTTP em 0.0.0.0.

## Condições de explorabilidade

Cliente Android/ambiente que permita HTTP, conexão direta conforme configuração e atacante capaz de observar/interferir no caminho. Plataformas que bloqueiem cleartext podem falhar em conectar. Proteções externas como VPN/gateway não foram inspecionadas.

## Evidência

frontend/lib/services/local/api\_ip\_service.dart:18–26

...

```
18: Future<String> getBaseUrl() async {
19: final savedIp = await getApiIpDigits();
20: final ip =
21: (isValidIp(savedIp) ? savedIp : null) ??
22: formatDigitsAsIp(savedIp) ??
23: formatDigitsAsIp(_defaultApiIpDigits) ??
24: '10.173.57.47';
25:
26: return 'http://$ip:$_apiPort';
...
```

frontend/lib/services/api/api\_service.dart:30–32

...

```
30: final response = await http
31: .post(uri, headers: headers, body: jsonEncode(body))
32: .timeout(const Duration(seconds: 10));
...
```

frontend/android/app/src/main/AndroidManifest.xml:13–15

...

```
13: android:icon="@mipmap/ic_launcher"
14: android:enableOnBackInvokedCallback="true"
15: android:usesCleartextTraffic="true">
...
```

## Impacto

Um observador no caminho de rede lê senhas e dados de saúde; um intermediário ativo pode adulterar respostas e capturar credenciais. Não foi executada interceptação de tráfego.

## Sugestão de correção

Configurar endpoint HTTPS validado e rejeitar esquema HTTP em release. Desabilitar `usesCleartextTraffic` na configuração de produção; eventual exceção local deve existir somente em debug. Manter validação normal de certificado e configurar TLS na terminação publicada.

## Critérios de aceite

- [ ] Release rejeita base HTTP e usa HTTPS para login, CRUD e exportação.
- [ ] Manifesto release desabilita cleartext; exceção debug não vaza para release.
- [ ] Certificado inválido é rejeitado; teste confirma ausência de bypass.

--- FIM ISSUE 8 ---